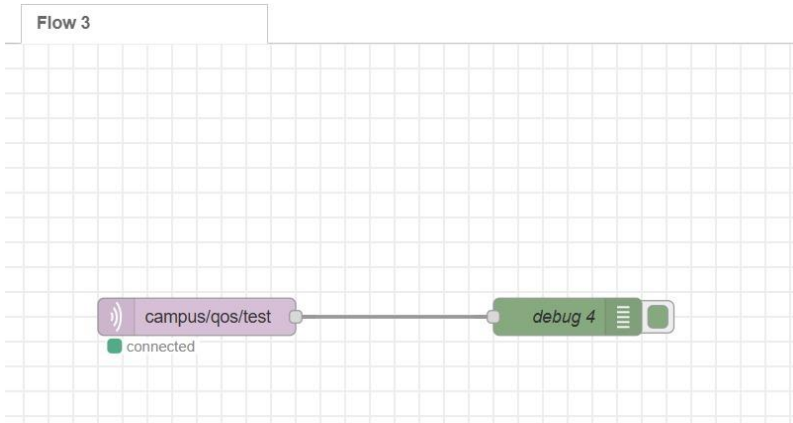


Exercise 5: MQTT Reliability, QoS Handshakes, and Retained Messages

Step 1: Role Reversal - Configuring Node-RED as the Subscriber



Step 2: Wireshark Initialization & QoS 0 (Fire and Forget)

mqtt://localhost:1883

V5 mqtt://localhost:1883

Docs Message Topics Authorization Properties Last Will Settings

1 { "test": "QoS 0 execution" }

JSON Retain QoS: 0 campus/qos/test Send

Response

Messages Visualization

Search All messages

campus/qos/test ("test": "QoS 0 execution") 01:33:57.127

Capturing from Adapter for loopback traffic capture

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.port == 1883

No.	Time	Source	Destination	Protocol	Length	Info
1054	147.626904800	127.0.0.1	127.0.0.1	TCP	44	64784 → 1883 [ACK] Seq=7 Ack=54 Win=255 Len=0
1318	162.640133800	127.0.0.1	127.0.0.1	TCP	45	[TCP Keep-Alive] 1883 → 64784 [ACK] Seq=53 Ack=7 Win=255 Len=1
1319	162.640133600	127.0.0.1	127.0.0.1	TCP	45	[TCP Keep-Alive] 64784 → 1883 [ACK] Seq=6 Ack=54 Win=255 Len=1
1320	162.640170900	127.0.0.1	127.0.0.1	TCP	56	[TCP Keep-Alive ACK] 1883 → 64784 [ACK] Seq=54 Ack=7 Win=255 Len=0 SLE=6 SRE=7
1321	162.640171400	127.0.0.1	127.0.0.1	TCP	56	[TCP Dup ACK 1054#1] 64784 → 1883 [ACK] Seq=7 Ack=54 Win=255 Len=0 SLE=53 SRE=54
1352	164.785189600	:::1	:::1	MQTT	66	Ping Request
1353	164.785277200	:::1	:::1	TCP	64	1883 → 52422 [ACK] Seq=5 Ack=7 Win=252 Len=0
1354	164.786656800	:::1	:::1	MQTT	66	Ping Response
1355	164.786717700	:::1	:::1	TCP	64	52422 → 1883 [ACK] Seq=7 Ack=7 Win=164 Len=0
1450	177.654218700	127.0.0.1	127.0.0.1	TCP	45	[TCP Keep-Alive] 1883 → 64784 [ACK] Seq=53 Ack=7 Win=255 Len=1
1451	177.654217700	127.0.0.1	127.0.0.1	TCP	45	[TCP Keep-Alive] 64784 → 1883 [ACK] Seq=6 Ack=54 Win=255 Len=1
1452	177.654268300	127.0.0.1	127.0.0.1	TCP	56	[TCP Keep-Alive ACK] 64784 → 1883 [ACK] Seq=7 Ack=54 Win=255 Len=0 SLE=53 SRE=54
1453	177.654277000	127.0.0.1	127.0.0.1	TCP	56	[TCP Dup ACK 1052#1] 1883 → 64784 [ACK] Seq=54 Ack=7 Win=255 Len=0 SLE=6 SRE=7
1509	184.611924400	:::1	:::1	MQTT	111	Publish Message [campus/qos/test]
1510	184.611974600	:::1	:::1	TCP	64	1883 → 56149 [ACK] Seq=7 Ack=101 Win=254 Len=0
1511	184.613620700	127.0.0.1	127.0.0.1	MQTT	90	Publish Message [campus/qos/test]
1512	184.613667400	127.0.0.1	127.0.0.1	TCP	44	64784 → 1883 [ACK] Seq=7 Ack=100 Win=255 Len=0

> Frame 1509: Packet, 111 bytes on wire (888 bits), 111 bytes captured (888 bits) on interface \Device\NPF_{...} id 0000 18 00 00 00 60 0d 15 5d 00 43 06 80 00 00 00 00 ...C.....

> Null/Loopback 0010 00 00 00 00 00 00 00 00 00 00 00 01 00 00 00 00U:[

> Internet Protocol Version 6, Src: ::1, Dst: ::1 0020 00 00 00 00 00 00 00 00 00 00 00 01 db 55 07 5b ...c:[q.I P.....

> Transmission Control Protocol, Src Port: 56149, Dst Port: 1883, Seq: 54, Ack: 7, Len: 47 0030 3d cc 63 92 5b 71 d8 49 50 18 00 ff c9 99 00 00 ...camp us/qos/t

> MQTT Telemetry Transport Protocol, Publish Message 0040 30 2d 00 0f 63 61 6d 70 75 73 2f 71 6f 73 2f 74 00...est:{"te st": "Qo

0050 65 73 74 00 7b 22 74 65 73 74 22 3a 20 22 51 6f S 0 exec ution")

0060 53 20 30 20 65 78 65 63 75 74 69 6f 6e 22 7d

24/04/2026, 01:33:57 node: debug 4

campus/qos/test : msg : Object

{ topic: "campus/qos/test", payload: object, qos: 0, retain: false, _msgid: "25a8eb1eac931d3e" }

Step 3: Verifying QoS 1 (At Least Once) and QoS 2 (Exactly Once)

mqtt://localhost:1883

V5

mqtt://localhost:1883

Disconnect

DocsMessageTopicsAuthorizationPropertiesLast WillSettings

1{"test": "QoS 1 execution"}

JSON

...

☐ Retain

QoS: 1

campus/qos/test

Send

Response

Not subscribed to any topics

Connected

...

Messages

Visualization

Search

All messages

🗑

↑

campus/qos/test

{"test": "QoS 1 execution"}

01:39:37.127

▼

mqtt.topic == "campus/qos/test"

No.	Time	Source	Destination	Protocol	Length	Info
174	20.800545900	::1	::1	MQTT	111	Publish Message [campus/qos/test]
1509	184.611924400	::1	::1	MQTT	111	Publish Message [campus/qos/test]
1511	184.613620700	127.0.0.1	127.0.0.1	MQTT	90	Publish Message [campus/qos/test]
1742	213.418014300	::1	::1	MQTT	111	Publish Message [campus/qos/test]
1744	213.419621300	127.0.0.1	127.0.0.1	MQTT	90	Publish Message [campus/qos/test]
4825	553.404703600	::1	::1	MQTT	113	Publish Message (id=2319) [campus/qos/test]
4829	553.423950200	127.0.0.1	127.0.0.1	MQTT	92	Publish Message (id=1) [campus/qos/test]

(Test QoS 1)

mqtt://localhost:1883

V5

mqtt://localhost:1883

Disconnect

DocsMessageTopicsAuthorizationPropertiesLast WillSettings

1{"test": "QoS 1 execution"}

JSON

...

☐ Retain

QoS: 1

campus/qos/test

Send

Response

Not subscribed to any topics

Connected

...

Messages

Visualization

Search

All messages

🗑

↑

campus/qos/test

{"test": "QoS 1 execution"}

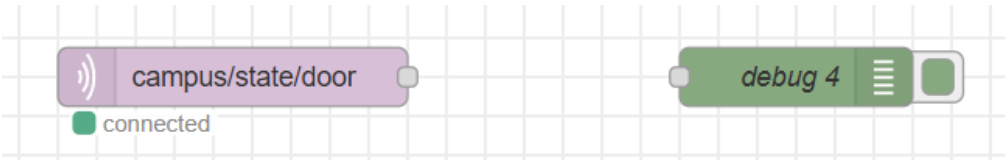
01:39:37.127

▼

6507	768.916523200	::1	::1	MQTT	113	Publish Message (id=2320) [campus/qos/test]
6509	768.919579900	::1	::1	MQTT	70	Publish Received (id=2320)
6511	768.919968900	127.0.0.1	127.0.0.1	MQTT	92	Publish Message (id=2) [campus/qos/test]
6515	768.928787900	::1	::1	MQTT	70	Publish Release (id=2320)
6517	768.930287600	127.0.0.1	127.0.0.1	MQTT	48	Publish Release (id=2)
6519	768.930803900	::1	::1	MQTT	70	Publish Complete (id=2320)

(Test QoS 2)

Step 4: The Retain Flag Mechanism



Disconnect the Subscriber

mqtt://localhost:1883

V5 mqtt://localhost:1883 Client ID Disconnect

Docs Message Topics Authorization Properties Last Will Settings

1 { "status": "OPEN" }

JSON Retain QoS: 0 campus/state/door Send

Response Not subscribed to any topics Connected

Messages Visualization

Search All messages

↑ campus/state/door {"status": "OPEN"} 01:50:12.406

Publish a Retained Message

24/04/2026, 01:51:09 node: debug 4
campus/state/door : msg : Object
 { topic: "campus/state/door", payload: object, qos: 0, retain: true, _msgid: "f95d4b051759efe2" }

Reconnect the Subscriber & Observe the Magic

Time	Source	Destination	Protocol	Message
11074	1245.4222002...	127.0.0.1	MQTT	68 Subscribe Request (id=24810) [campus/state/door]
11076	1245.4248693...	127.0.0.1	MQTT	49 Subscribe Ack (id=24810)
11078	1245.4270689...	127.0.0.1	MQTT	83 Publish Message [campus/state/door]
11135	1248.7060483...	:::1	MQTT	66 Ping Request
11137	1248.7070882...	:::1	MQTT	66 Ping Response
11747	1304.9495008...	:::1	MQTT	66 Ping Request
11749	1304.9515039...	:::1	MQTT	66 Ping Response
11751	1305.4437393...	127.0.0.1	MQTT	46 Ping Request
11753	1305.4449888...	127.0.0.1	MQTT	46 Ping Response

> Frame 11078: Packet, 83 bytes on wire (664 bits), 83 bytes captured (664 bits) on interface \Device\NPF_{...} id 0

> Null/Loopback

> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

> Transmission Control Protocol, Src Port: 1883, Dst Port: 56962, Seq: 10, Ack: 62, Len: 39

> MQ Telemetry Transport Protocol, Publish Message

Header Flags: 0x31, Message Type: Publish Message, QoS Level: At most once delivery (Fire and Forget), Retain

0011 = Message Type: Publish Message (3)

.... 0... = DUP Flag: Not set

.... 00.. = QoS Level: At most once delivery (Fire and Forget) (0)

.... ...1 = Retain: Set

Msg Len: 37

Topic Length: 17

Topic: campus/state/door

Message: 7b22737461747573223a20224f50454e227d

0000 02 00 00 00 45 00 00 4f 2d aa 40 00 80 06 ce fc ...E..0..@....
0010 7f 00 00 01 7f 00 00 01 07 5b de 82 94 52 fb b2[...R..
0020 64 90 ad 05 50 18 00 ff 91 e2 00 00 31 25 00 11 d...P...1%..
0030 63 61 6d 70 75 73 2f 73 74 61 74 65 2f 64 6f 6f campus/s tate/doo
0040 72 7b 22 73 74 61 74 75 73 22 3a 20 22 4f 50 45 r{"statu s": "OPE
0050 4e 22 7d N"}
0050

Wireshark Analysis

Part 5: Discussion & Architectural Analysis

1. If you are designing a battery powered ESP32 temperature sensor that updates every 10 seconds, which QoS level would you choose and why? What are the mathematical consequences on battery life and bandwidth if you incorrectly select QoS 2 for this scenario?

For a battery-powered sensor updating every 10 seconds, QoS 0 is the most efficient choice.

This is because: -

- **Energy Efficiency:** Every packet sent requires the ESP32 to keep its Wi-Fi radioactive. Since QoS 0 is a "fire and forget" method, the radio can power down immediately after a single transmission.
- **Nature of the Data:** Temperature data is "transient." If one reading is lost, a new one arrives just 10 seconds later. The cost of losing a single data point is lower than the cost of the battery drain required to guarantee its arrival.

Mathematical Consequences of Selecting QoS 2

Bandwidth Consumption:

QoS 2 requires a 4-packet handshake (PUBLISH, PUBREC, PUBREL, PUBCOMP). This increases the control-plane traffic by **400%** compared to QoS 0. In a 24-hour period (8,640 updates), you would send 34,560 packets instead of 8,640.

2. Explain the fundamental difference between standard MQTT message delivery and a Retained message. Give a real-world Industrial IoT (IIoT) scenario where a Retained message is mandatory for system safety when a new client connects to the broker.

The Fundamental Difference

- **Standard Message:** The broker receives a message and forwards it only to clients currently subscribed to that topic. Once delivered, the message is discarded by the broker. A client that connects after the message was sent will see nothing until the next update.
- **Retained Message:** The broker stores the last known good message for a specific topic. When a new client subscribes to that topic, the broker immediately pushes that "retained" message to the client, providing instant context without waiting for the next sensor update.

IIoT Scenario: System Safety & Emergency Stop (E-Stop)

In an industrial environment, **Retained Messages** are mandatory for **Safety Status** or **System Interlocks**.

Scenario: Imagine a central control dashboard (Client B) that connects to a broker to monitor a high-pressure steam valve. If the valve is currently in an "EMERGENCY SHUTDOWN" state, that status must be a Retained Message.

Why it's mandatory for safety:

If the dashboard loses power and reboots, it must know the state of the valve immediately upon reconnection. Without a Retained Message, the dashboard might default to showing a "Safe/Normal" status (or no status at all) for the 10 or 30 seconds it takes for the valve sensor to send its next heartbeat. In that window, an operator might mistakenly attempt to start a process, leading to a catastrophic failure because they didn't know the system was in an emergency state.